Lecture 3 in JS (CSTD 6)

Sunday, March 30, 2025 5:47 PM

If you have an object like:

var obj = {id:3,name: "mohammad",address:"cairo"}

==> To get an item from object you have to use two method.

1) Using (.) --Ex--> [obj.id | obj.name | obj.address]
2) Using indexer --Ex--> [obj['id'] | obj['name'] | obj['address']

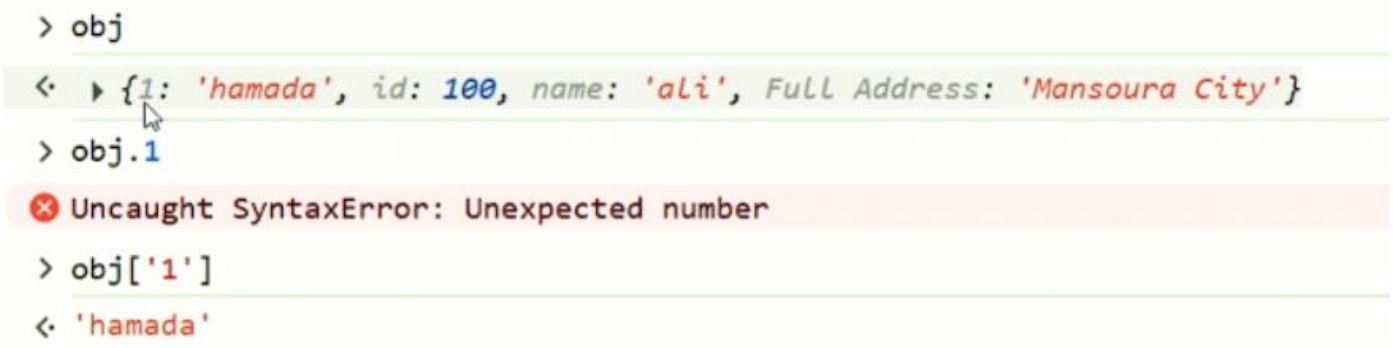
*** But…… which one is the best…?

See this example.



==> this example say that if the key contain of two words you can't used (.) to get it's value but…
==> you can use indexer .

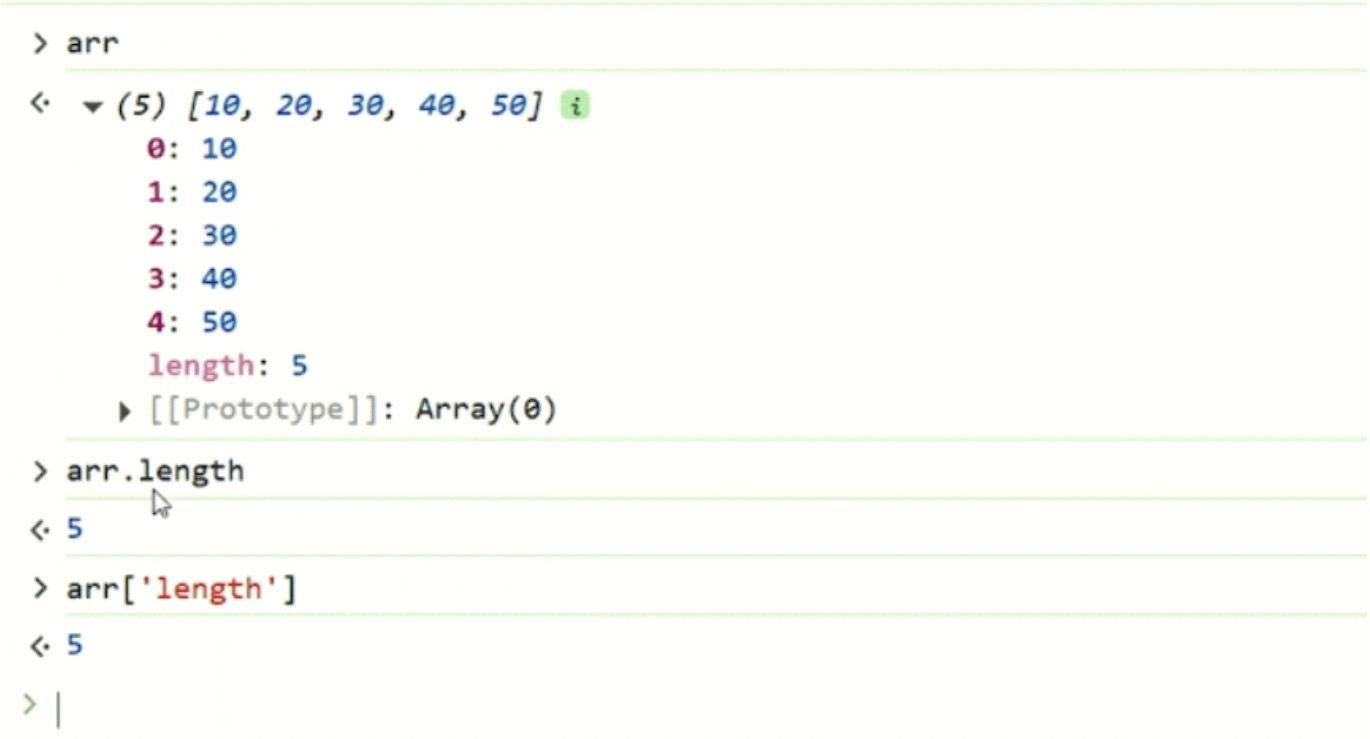
See this example too.



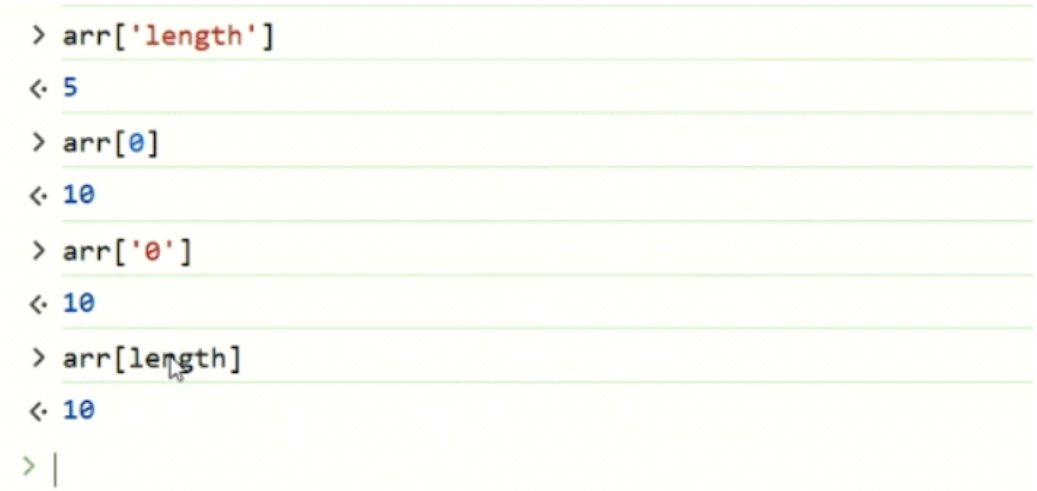
==> this example say that is the key is number you can't also use (.) to call it's value but..
==> you can use indexer.

Which mean that using indexer is better than using (.)

Note that : array is similar to object.



=> this image show that array is an object with a key of type number and contain kay call length in the end .



===> Associative array

Ex)

```
var arr = [];// dense array
arr.ID = 500;
arr.Name = "A";
```

==>This way is not good because the length of the array sill 0
==> so if you want to add string key you should use object not array.

===> for in , for of

=: for in ---> return the keys for the objects or array
=: for of ---> return the values for the objects for array.

Ex) **for in**

```
var obj = {
  id: 100, name: "ali", 'Full Address': "Mansoura City", 1: "hamada", myfun: function () { console.log('hi')}
};
  for (var i in obj) {
  console.log(i+ " : "+obj[i]);
}
```

==> j in last loop equal to the kays [id, name, full Address, myfun]

Ex) **for of**

```
var obj = {
  id: 100, name: "ali", 'Full Address': "Mansoura City", 1: "hamada", myfun: function () { console.log('hi')}
};
  for (var i of obj) {
  console.log(i+ " : "+obj[i]);
}
```

==> j in last loop equal to the values ['ali', 'Mansoura', 'hamada', function]

==> String

==> you can define string by two ways:
1- var str = 'value'
2- var str = new String('value')
==> the string is an array of characters. So you can use the indexer to get the character, but you can't change the character by the indexer.
==> string have a property length to return the size of the array.

Ex)

```
> str
< 'ITI Tanta Branch'
> str.length
< 16
> str[15]
< 'h'
> str[0]
< 'I'
```

```
> str[0]='T'
<· 'T'
> str
<· 'ITI Tanta Branch'
```

nothing change

```
> str="Front and cross mobile"
<· 'Front and cross mobile'
> str
<· 'Front and cross mobile'
```

String Function:

1) **charAt()**

==> use to return a character using the index of the character.
==> it's better than using the indexer because the indexer return undefined if you passed unexcited index.
(out of range index)

```
> str
<· 'ITI Tanta Branch'
> str.length
<· 16
> str[16]
<· undefined
>
```

==> if you pass an out of range index the fun will return an empty string ['']
==> if you use + before calling the fun, It will return 0
==> **note that** : adding + before the string will convert it to a number.

```
'ITI Tanta Branch'
> str.charAt(0)
'I'
> str[0]
'I'
> str.charAt(160)
''
> str.charAt(160).length
0
> +str.charAt(160)
0
```

---

2) indexOf()

==> will return the first index of the passed character.

==> you can used it to pass to check if the string contain a character or no, because if the string didn't contain the character the fun will return -1.

==> indexOf() can take two parameters:

1- The character which you want to search about it.

2- The index which you want to start from it.

3)

```
> str
'ITI Tanta Branch'
> str.indexOf('T')
1
> str.lastIndexOf('T')
4
```

```
> "nasrnasr.com".indexOf('@')
-1
> "nasr@nasr.com".indexOf('@')
4
> "nasr@nasr.com".indexOf('@',3)
4
```

f(searchString, ?position)

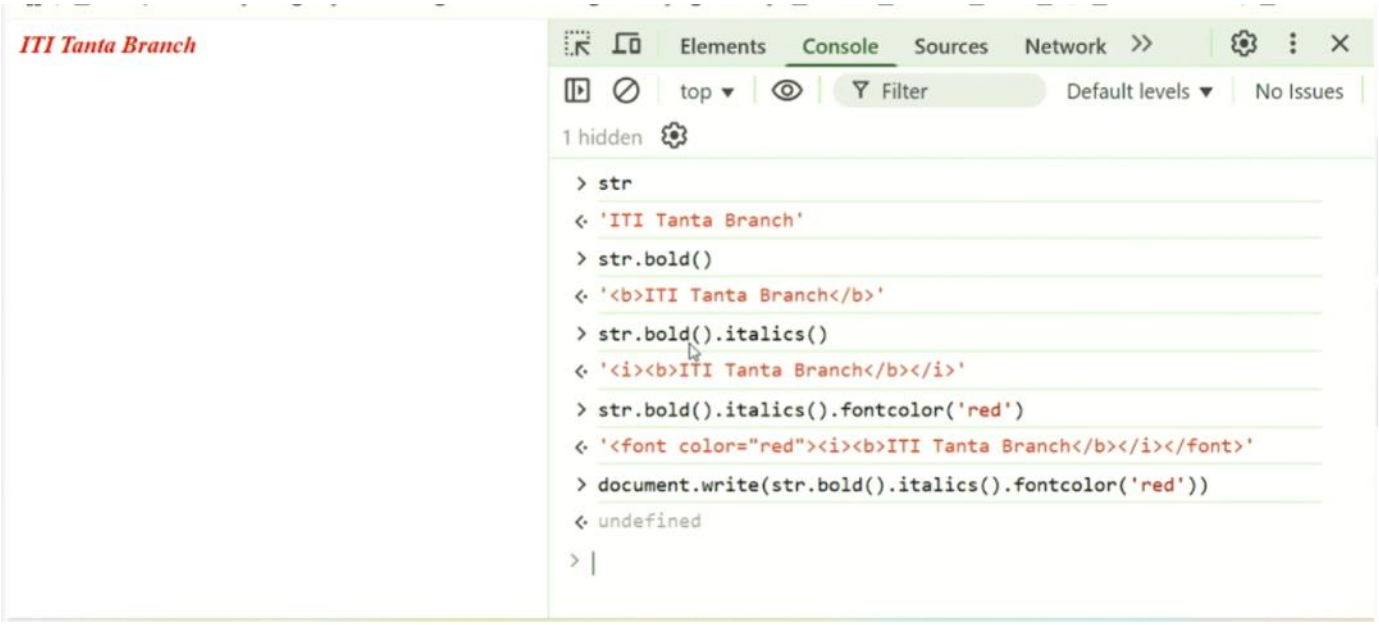
---

3) replace(), replaceAll(), toUpperCase()

```
'ITI Tanta Branch'
> str.replace("T","$")
'I$I Tanta Branch'
> str.replaceAll("T","$")
'I$I $anta Branch'
> str.toUpperCase()
'ITI TANTA BRANCH'
> str.toUpperCase().replaceAll("T","$")
'I$I $AN$A BRANCH'
> str
'ITI Tanta Branch'
>
```

---

4) bold(), italics(), fontcolor()



---

5) split()

==> used to convert the string to array.
==> it take a value which used to detect the end of the divided items.
==> if you call split("") and passing an empty string, the string will divided to array of character.

```
> "mohammad ramadan".split()
< ▶ ['mohammad ramadan']
> "mohammad ramadan".split("")
< ▶ (16) ['m', 'o', 'h', 'a', 'm', 'm', 'a', 'd', ' ', 'r', 'a', 'm', 'a', 'd', 'a', 'n']
> "mohammad ramadan".split("")
< ▶ (16) ['m', 'o', 'h', 'a', 'm', 'm', 'a', 'd', ' ', 'r', 'a', 'm', 'a', 'd', 'a', 'n']
> "mohammad ramadan".split(" ")
< ▶ (2) ['mohammad', 'ramadan']
> "mohammad ramadan".split("%")
< ▶ ['mohammad ramadan']
```

==> you can also passed a limit which determine the number of returned elements.
==> **note That:** you can also pass a regular expression.

Ex)

```
> "iti tanta mansoura cairo smart newcapital".split(" ")
< ▶ (6) ['iti', 'tanta', 'mansoura', 'cairo', 'smart', 'newcapital']
> "iti tanta mansoura cairo smart newcapital".split(" ",3)
< ▶ (3) ['iti', 'tanta', 'mansoura']
> "iti tanta mansoura cairo smart newcapital".split(" ",2)
< ▶ (2) ['iti', 'tanta']
> "iti tanta mansoura cairo smart newcapital".split(" ",1)
< ▶ ['iti']
```

---

6) substring()

==> substring(start, end)
==> used to take substring from string and copy it to another variable.
==> if you didn't pass any thing the fun will return all the string.

```
> str.substring(0,3)
<- 'ITI'
> str
<- 'ITI Tanta Branch'
> str.substring()
<- 'ITI Tanta Branch'
> str.substring(0)
<- 'ITI Tanta Branch'
>
```

==> if the start parameter is less than the end parameter the fun can hand this problem.
==> if you pass a negative value the fun will convert it to zero.

```
> str.substring(4,0)
<- 'ITI '
> str.substring(0,4)
<- 'ITI '
> str.substring(-1,3)
<- 'ITI'
> str.substring(0,3)
<- 'ITI'
> str.substring(3,-1)
<- 'ITI'
```

---

7) substr()

==> substr(first index, the length of the returned string).
==> it's not equal to substring()
==> it take start index and number of indexes which you want to add it to the string.

```
> str
<- 'ITI Tanta Branch'
> str.substr(0,3)
<- 'ITI'
> str.substring(0,3)
```

```
> str.substring(0,3)
'ITI'
> str.substr(4,5)
'Tanta'
> str.substring(4,9)
'Tanta'
```

==> If you passed a negative value like (-5), it's mean that you want the fun to start form the last fifth item

```
> str.substring(-6)
'ITI Tanta Branch'
> str.substr(-6)
'Branch'
> str.substr(-6,3)
'Bra'
> str.substr(-9,5)
'ta Br'
```

---

8) splice()

==> splice(first index , last index)

==> it's work like substring but accept negative number like subtr.

==> if the first index is bigger than the last index the function will return empty string ['']

```
> str
'ITI Tanta Branch'
> str.slice()
'ITI Tanta Branch'
> str.slice(0,3)
'ITI'
> str.slice(-6,-3)
'Bra'
```

Boolean

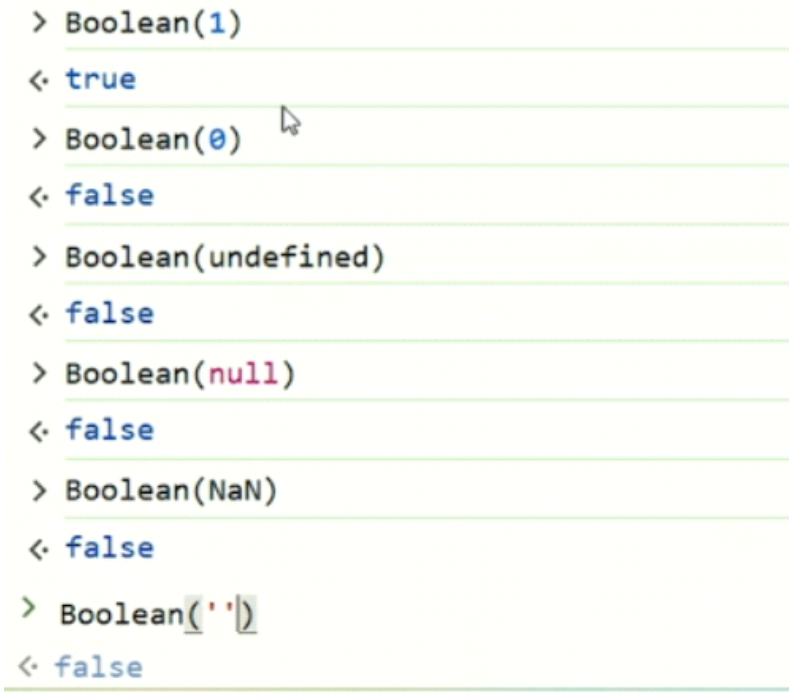
==> to define boolean variable use…

1- Literal way: var test = true
2- Constructor: var text = new Boolean(); // false

falsy and truthy values:

Falsy values : [undefined, null, NaN, 0, false, '']
Truthy values : is any value else.

==> to check if the value is true or false use: Boolean()
==> it used in if statement condition.



Logical && and ||

&&:

==> if : the first item is true it will return the second item.
==> if : the first item is falsy it will return it.

```
> "Ahmed"&&20
< 20
> "Ahmed"&&0
< 0
> 0&&"Ahmed"
< 0
```

||:

==> it will return the last truthy item

```
> "ahmed" || 0
< 'ahmed'
> undefined || "hamada"
< 'hamada'
> undefined || 0
< 0
> undefined || false
< false
> undefined || "false"
< 'false'
```

|| and &&

==> not always used in if statement.

```
var userinput = prompt("Enter name") || "Default Name";
```

Date:

==> to define date use --> var today = new Date();

```
> today
< Mon Mar 17 2025 14:55:15 GMT+0200 (Eastern European Standard Time)
> today.getFullYear()
< 2025
> today.getYear()
< 125
> today.getDate()
< 17
> today.getDay()
< 1
```

2025 − 1990

from − Monday

To change the date:

```
> today
< Mon Mar 17 2025 14:59:25 GMT+0200 (Eastern European Standard Time)
> today.setFullYear(2026)
< 1773752365039
> new Date(1773752365039)
< Tue Mar 17 2026 14:59:25 GMT+0200 (Eastern European Standard Time)
>
```

Deal with local:

```
< Mon Mar 17 2025 15:01:15 GMT+0200 (Eastern European Standard Time)
> today.toTimeString()
< '15:01:15 GMT+0200 (Eastern European Standard Time)'
> today.toLocaleTimeString()
< '3:01:15 PM'
> today.toLocaleDateString()
< '3/17/2025'
> today.toLocaleDateString("ar-EG")
< '٣/١٧/٢٠٢٥'
> today.toLocaleDateString("ar-SA")
< '١٧/٩/١٤٤٦ هـ'
```

Math function:

Note that :

[+] before string --> use to covert string to number.

```
> Math.PI
<· 3.141592653589793
> (Math.PI).toFixed(2)
<· '3.14'
> +(Math.PI).toFixed(2)
<· 3.14
>
```

```
> Math.round(3.56)
<· 4
> Math.round(3.46)
<· 3
> Math.ceil(3.46)
<· 4
> Math.ceil(3.1)
<· 4
> Math.floor(3.9)
<· 3
```

Rondom:

```
> Math.random()*10
< 5.1520063927704545
> Math.floor(Math.random()*10)
< 8
> Math.floor(Math.random()*10)
< 4
> Math.floor(Math.random()*10)
< 1
> Math.floor(Math.random()*10)
```

```
> Math.floor(Math.random()*11)
< 0
> Math.floor(Math.random()*100)
< 30
> Math.floor(Math.random()*100)
< 44
> Math.floor(Math.random()*101)
< 15
> Math.floor(Math.random()*101)
< 3
```

Mean less than 11

This JavaScript function always returns a random number between min and max (both included):

Example

```
function getRndInteger(min, max) {
  return Math.floor(Math.random() * (max - min + 1) ) + min;
}
```